

VLA-Perf (Jiang et al., 2026)

1. Citation & Source Metadata

- **Document Identifier:** WEEK 11_PAPER1_2602.18397v1.pdf
- **Full Title:** *How Fast Can I Run My VLA? Demystifying VLA Inference Performance with VLA-Perf*
- **Authors:** Wengi Jiang, Jason Clemons, Karu Sankaralingam, Christos Kozyrakis (NVIDIA Research)
- **Publication:** arXiv:2602.18397v1 [cs.RO] (20 Feb 2026)
- **Repository:** github.com/NVlabs/vla-perf

2. Core Technical Findings & Methodology

- **Performance Tool (VLA-Perf):** Introduced a roofline-based analytical performance model that estimates optimal inference latency and throughput across arbitrary combinations of VLA architectures, model scales, hardware backends, and network topologies without full physical deployment.
- **Real-Time Thresholds Defined:** Established 10 Hz (100 ms latency) as the minimum acceptable rate for camera signal ingestion and 100 Hz (10 ms latency) as the target for high-performance physical control loops.
- **314 Hz Peak Achievability:** Demonstrated that small diffusion-based VLA models (π_0) can reach up to 314.4 Hz (3.18 ms latency) throughput on top-tier datacenter hardware (NVIDIA B100), proving that high-frequency action generation is technically achievable under optimal hardware conditions.
- **Hardware Bottlenecks:**
 - a) Vision Encoders and VLM Backbones are **Compute-Bound** on desktop and datacenter GPUs due to high operator intensity (token batching).
 - b) Action Experts are **Memory-Bound** because they operate on small token lengths (action chunk sizes).
 - c) Mobile embedded platforms (e.g., Jetson Thor) become memory-bound across all components due to LPDDR bandwidth constraints (270 GB/s).
- **Architecture Comparison:** Diffusion-based action experts with action chunking execute $100 \times$ **faster** (3.2 ms) than vanilla step-by-step autoregressive models (327.6 ms).
- **Deployment Warnings:**
 - a) **Device-Server Collaboration:** Splitting workloads (VLM on server, Action Expert on robot) is slower than server-only or device-only execution due to the massive network transfer latency of downloading the VLM's KV Cache.
 - b) **Cloud Latency:** Synchronous cloud-server inference cannot achieve 100 Hz due to round-trip ping times (> 10 ms per transfer). Achieving ≥ 100 Hz throughput over cloud backends requires **Asynchronous Execution**.

3. Target Citation Claims for HRAF Paper (Task Sheet Specific)

"[System-1 / System-2 Framework]:

Decoupling slow, high-level vision-language reasoning (System-2) from fast, low-level reactive motor control (System-1) is essential for real-time robotic performance [VLA-Perf, 2026]. Running a VLM synchronously inside a continuous control loop introduces severe latency bottlenecks (2--5 Hz or up to 52 ms on edge chips), causing thread stalls and unstable motor execution."

"[314 Hz Achievability & ADCA Target Motivation]:

Benchmark analyses confirm that low-level action policies can achieve execution frequencies up to 314 Hz (3.18 ms) on high-performance accelerators [VLA-Perf, 2026]. This proves that sub-10 ms real-time control is technically achievable and directly motivates the 100 Hz--300 Hz control frequency targets enforced by the ADCA framework."

"[Asynchronous Synchronization]:

Furthermore, device-server partitioning schemes that offload VLM backends while running local action experts incur severe network transmission overheads when synchronizing large Key-Value (KV) caches [VLA-Perf, 2026]. To sustain high-frequency control without network bottlenecks, high-speed action generation requires decoupled asynchronous state synchronization."

4. ADCA / HARM Architecture Mapping

- **Justifies System-1 / System-2 Framework (ADCA):**
 - Directly cites VLA-Perf to justify splitting ADCA into a fast, local **System-1 execution loop** (100 Hz--300 Hz) for inverse kinematics/telemetry and a decoupled background **System-2 VLM planner** via an asynchronous StateBridge mailbox queue.
- **Motivates ADCA Frequency Targets:**
 - Uses the empirical proof of 314.4 Hz **VLA achievability** on B100 hardware as the benchmark baseline to justify targeting ≥ 100 Hz real-time control loops in the ADCA architecture.
- **Validates Thread Offloading via asyncio (PEP 3156):**
 - Confirms VLA-Perf's finding that asynchronous execution overlaps network upload/download latency with motor execution, preventing event loop blocking.
- **Supports Safety Fallback (HARM / IEC 61508):**
 - Proves that high-level VLM pipelines cannot guarantee deterministic sub-10 ms response times. This justifies HARM's requirement to enforce local, deterministic joint/velocity limit checks and default to a HOLD state whenever high-level guidance stalls.

5. Summary Table: 15 Key Takeaways Reference

#	Domain	Key Finding / Takeaway
1	Baseline	Datacenter GPUs (B100) run small VLAs (π_0) at 61--314 Hz; edge GPUs (Jetson Thor) reach only 19 Hz.
2	Bottlenecks	Vision/VLM are Compute-Bound; Action Experts are Memory-Bound (LPDDR RAM limits edge chips).
3	Scaling	Latency grows linearly (1: 1) with parameter scaling.
4	Model Limits	Datacenter GPUs can run massive 81B models at real-time speeds (9.6 Hz).
5	Context	Datacenter GPUs handle 1,000 past visual context frames; edge GPUs cap out around 100 steps.
6	Parameters	Increasing diffusion denoising steps drastically increases latency; increasing action chunk size barely affects delay.
7	Architecture	Diffusion VLAs are 1--2 orders of magnitude faster than vanilla autoregressive VLAs.
8	Autoregressive	Autoregressive models are competitive only with tiny action spaces (≤ 7 DoF) or parallel decoding.
9	Placement	Server-side execution over Wi-Fi/5G outperforms on-board edge execution in almost all cases.
10	Hybrid Split	Device-Server hybrid partitioning is slower than single-node execution due to KV Cache download lag.
11	Asynchrony	Asynchronous inference improves server-side throughput by $2.6 \times$ -- $6 \times$ over wireless connections.
12	Dual-System	Async System 1 + System 2 pipelines give up to $2.24 \times$ speedups over fast wired networks.
13	100 Hz On-Device	Reaching 100 Hz on edge GPUs requires model compression, fewer diffusion steps, or quantization.
14	100 Hz Edge Server	Reaching 100 Hz on edge servers requires datacenter-class GPUs (B100) and Wi-Fi 7 / 10G Ethernet.
15	100 Hz Cloud	Reaching 100 Hz over cloud servers strictly requires asynchronous execution to bypass ping delays.

VLMPC (Zhao et al., 2024)

1. Citation & Source Metadata

- **Document Identifier:** VLMPC_Zhao_et_al_2024.pdf
- **Full Title:** *VLMPC: Vision-Language Model Predictive Control for Robotic Manipulation*
- **Authors:** Wentao Zhao*, Jiaming Chen*, Ziyu Meng, Donghui Mao, Ran Song†, Wei Zhang (Shandong University)
- **Publication:** IEEE Robotics and Automation Letters (RA-L) / arXiv:2403.11182 (2024)
- **Repository:** github.com/PPjmchen/VLMPC

2. Core Technical Findings & Methodology

- **Architectural Paradigm:** Integrates Vision-Language Models (VLMs) into a Model Predictive Control (MPC) loop to combine open-world semantic perception with forward-looking trajectory optimization.
- **Conditional Action Sampling:** Uses a VLM (2D bounding box prompts via GPT-4V/Qwen-VL) to propose high-level direction vectors $(dx, dy, dz, rx, ry, rz, g_t)$, which center a Gaussian sampling distribution to generate N candidate action sequences.
- **Action-Conditioned Video Prediction (DMVFN-Act):** Modifies a voxel-flow network (DMVFN) to rapidly generate N simulated future video clips (T steps into the future) conditioned on candidate action sequences in a single GPU batch.
- **Hierarchical Cost Function:** Evaluates simulated videos using a VLM Switcher (w_D) that dynamically balances:
 - **Pixel Distance Cost (C_P):** L_2 distance to goal image (effective for simple single-stage tasks).
 - **VLM-Assisted Cost (C_{VLM}):** Uses a VLM to identify sub-goal objects and obstacles, tracked across video frames via SiamRPN bounding boxes, to reward sub-goal distance while penalizing obstacle proximity.
- **Execution & Recycler:** Executes only the first optimal action step a_{t+1}^* , recycling remaining sequence steps (μ_{t+1}^{sub}) to inform sampling in the next time step.

3. Target Citation Claims for HRAF Paper

"Large-scale Vision-Language Models (VLMs) suffer from severe visual hallucination and semantic misinterpretation, making raw VLM outputs inherently unsafe as direct deployment controllers for physical robotics [VLMPC, 2024]. Left unmitigated, VLM hallucinations manifest as sudden direction flips, erratic motor outputs, and unexpected collisions."

"To mitigate VLM hallucinations, software-level frameworks like VLMPC attempt to smooth action outputs by performing weighted elementwise blending with historical candidate trajectories [VLMPC, 2024]. However, such mathematical smoothing lacks deterministic safety guarantees. In contrast, the High-assurance Action Real-time Monitor (HARM) enforces hardware-level containment by evaluating candidate actions against strict kinematic, velocity, and envelope boundaries prior to motor execution [HARM, 2026]."

4. Mitigation Comparison: VLMPC vs. HARM Framework

Dimension	VLMPC Mitigation Approach	HARM Framework Approach (Our Architectural Goal)
Mitigation Mechanism	Mathematical Trajectory Blending: Performs weighted elementwise sum ($\mu_t = w_{\text{VLM}}\mu_t^{\text{VLM}} + w_{\text{sub}}\mu_t^{\text{sub}}$) between VLM output and previous action sequence.	Deterministic Rule Envelope / Safety Guard: Enforces strict joint/velocity/acceleration boundaries and workspace bounding boxes independent of model logic.
Layer of Operation	Software Sampling Layer: Alters the mean of the Gaussian sampling distribution prior to video simulation.	Hardware / Execution Gateway: Sits directly above the low-level actuator drive thread.
Safety Assurance Level	Probabilistic / Heuristic: Reduces sudden jitter but cannot guarantee safety if both current VLM and historical actions carry errors.	Deterministic / Guaranteed (IEC 61508 compliant): Hard-blocks unsafe commands and forces immediate fallback state (HOLD / STOP).
Failure Response	Continuous execution with potentially degraded/skewed action trajectory.	Latency fault override or immediate boundary check trigger, returning system to safe baseline state.

5. ADCA / HRAF Architecture Mapping

- **Justifies Synchronous Bottleneck (ADCA Need):**
 - Authors explicitly acknowledge that calling a large-scale VLM and video simulation inside every MPC step imposes significant computational overhead, proving why System-2 VLM inference must be decoupled into background asynchronous worker threads.
- **Motivates Structural Safety (HARM Need):**
 - Proves that software-level prompt engineering and trajectory blending are insufficient to prevent physical collisions in tightly constrained workspaces. This directly justifies HARM's role as a standalone, deterministic safety watchdog.

- **Validates Dual-Layer Evaluation:**
 - Reinforces HRAF's core design pattern: utilizing high-level semantic reasoning for candidate generation while delegating verification to fast, localized execution engines.

PEP 3156 (van Rossum, 2012)

1. Citation & Source Metadata

- **Document Identifier:** WEEK11_PAPER3_asyncio PEP 3156.pdf
- **Full Title:** *PEP 3156 – Asynchronous IO Support Rebooted: the "asyncio" Module*
- **Author:** Guido van Rossum (Python Software Foundation)
- **Status / Date:** Final / Created December 12, 2012
- **Specification Source:** Python Enhancement Proposals (PEPs) / Python 3.4+ Standard Library

2. Core Technical Specifications & Architecture

- **Pluggable Single-Threaded Event Loop:** Specifies the fundamental `AbstractEventLoop` interface that manages execution contexts, non-blocking I/O polling, and callback queues within a single operating system thread.
- **Task Scheduling & Futures (`asyncio.Future` / `Task`):** Introduces non-blocking future objects and task wrappers that allow asynchronous coroutines to yield execution without blocking the event loop thread while waiting for network or I/O responses.
- **Executor Pool Offloading (`run_in_executor`):** Provides a standardized interface (`loop.run_in_executor`) to delegate heavy, synchronous, CPU-bound tasks or blocking C-extensions to background thread/process pools (`ThreadPoolExecutor`) without interrupting the primary event loop.
- **Mutual Exclusion of Callbacks:** Guarantees strict serialization of callbacks within the loop, preventing race conditions on shared memory without requiring complex multi-threading locks (Mutex).
- **Transport & Protocol Separation:** Decouples low-level network communication abstractions (sockets, pipes) from high-level application logic through non-blocking event-driven interfaces.

3. Target Citation Claims for HRAF Paper (Task Sheet Specific)

"[**Authoritative asyncio Reference**]: To prevent thread stalls, race conditions, and context-switching overheads in real-time execution loops, the ADCA framework leverages a single-threaded non-blocking event loop model grounded in the PEP 3156 specification [van Rossum, 2012]."

"[**Asynchronous Non-Blocking I/O**]: High-latency network operations—such as sending visual observations to remote System-2 VLM instances—are managed via non-blocking

coroutines and Futures [van Rossum, 2012], allowing the control pipeline to process continuous state updates without blocking on network round-trip ping delays."

"[**Thread Offloading via Executors**]: Heavy, CPU-bound computations (e.g., matrix transformations, Denavit-Hartenberg parameter evaluation, or safety constraint evaluations) are offloaded to background worker threads using executor pools (`run_in_executor`) [van Rossum, 2012]. This ensures that the primary System-1 motor execution thread continuously meets its sub-10 ms control frequency targets."

4. ADCA / HARM Architecture Mapping

- **Authoritative Architectural Reference for ADCA:**
 - Serves as the primary software engineering citation for ADCA's StateBridge mailbox queue and event loop concurrency model.
- **System-1 / System-2 Execution Decoupling:**
 - Cites PEP 3156 to justify how System-1 can continuously execute local motor telemetry commands at 100--300 Hz while asynchronously waiting for System-2 VLM planning payloads to resolve in the background.
- **Prevents Event Loop Blocking via `run_in_executor`:**
 - Directly supports ADCA's design requirement to isolate heavy kinematic calculations from the main execution thread using dedicated executor pools.
- **Ensures Deterministic Safety Monitoring (HARM):**
 - Proves that non-blocking asynchronous architectures prevent high-level VLM latency spikes from delaying safety monitoring threads, guaranteeing that HARM can intercept and override unsafe joint trajectory commands deterministically.

5. Summary Table: Core PEP 3156 Concepts Reference

#	Feature / Class	Core Architectural Function	ADCA / HRAF Application
1	AbstractEventLoop	Single-threaded engine for managing I/O callbacks and tasks.	Forms the core execution loop for real-time telemetry and state updates.
2	Future & Task	Non-blocking handles representing pending asynchronous results.	Tracks incoming VLM high-level action candidate proposals in the background.
3	<code>run_in_executor</code>	Offloads blocking/CPU-bound functions to thread pools.	Isolates heavy inverse kinematics and coordinate transformations from motor loops.

4	Mutual Exclusion	Serializes callback execution to prevent shared state corruption.	Ensures atomic state updates across the StateBridge without lock contention.
5	Transports & Protocols	Non-blocking abstraction layer for socket and inter-process I/O.	Manages low-latency IPC queues between local hardware controllers and cloud servers.

IEC 61508 (Functional Safety)

1. Citation & Source Metadata

- **Document Identifier:** WEEK11_PAPER3_IEC 61508 & Functional Safety-2022.pdf
- **Full Title:** *Overview of IEC 61508 & Functional Safety: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems*
- **Issuing Body:** International Electrotechnical Commission (IEC)
- **Standard Category:** Basic Functional Safety Standard (E/E/PE Safety-Related Systems)

2. Core Technical Concepts & Safety Principles

- **Functional Safety Definition:** Freedom from unacceptable risk depending on a system or equipment operating correctly in response to its input signals.
- **Safety Integrity Levels (SIL 1–4):** Quantified discrete levels specifying target risk reduction requirements. Higher SIL levels (SIL 4) correspond to higher risk reduction factors and lower acceptable failure rates.
- **Failure Classification:**
 - **Random Hardware Failures:** Arising from physical component degradation over time.
 - **Systematic Failures:** Built-in design, software, or specification errors (accounting for over 60% of control system failures).
- **Defence-in-Depth & Fail-Safe Defaults:** Risk-based safety functions implemented through independent input/controller/output loops to bring systems to a safe state upon error detection or limit violations.

3. Target Citation Claims for HRAF Paper (Task Sheet Specific)

"[**Safety-Critical Design Principles Applied to HARM**]: To guarantee functional safety in physical human-robot environments, the High-assurance Action Real-time Monitor (HARM) incorporates the core safety-critical principles of IEC 61508 [IEC 61508, 2022]. HARM establishes an independent, deterministic execution barrier that operates above the motor control loop, insulating physical actuators from systematic software errors or unverified VLM outputs."

"[**Defence-in-Depth Architecture**]: Following IEC 61508 defence-in-depth recommendations [IEC 61508, 2022], HARM executes multi-layered verification—

evaluating joint position limits, acceleration thresholds, and workspace bounding envelopes independently of the high-level VLM task planner."

"**[Fail-Safe Default Enforcement]**: When latency spikes, thread stalls, or boundary violations occur, HARM enforces fail-safe default states (such as immediate velocity damping or zero-torque HOLD modes) to prevent physical hazards, adhering to the safety integrity performance requirements defined in IEC 61508 [IEC 61508, 2022]."

4. ADCA / HARM Architecture Mapping

- **Establishes HARM as a Dedicated Safety Guard:** Cites IEC 61508 to justify why HARM sits directly between high-level controllers and lower-level actuator drivers as an E/E/PE safety-related logic solver.
- **Mitigates Systematic VLM / Software Failures:** Provides standard-backed reasoning for why software-level prompt engineering or trajectory smoothing (e.g., in VLMPC) is insufficient, necessitating deterministic, hardware-adjacent rule checking.
- **Defines Target Safety States:** Directly justifies HARM's immediate fallback behavior (HOLD / velocity limit override) whenever high-level asynchronous VLM threads stall or exceed time-to-react budgets.

5. Summary Table: Core IEC 61508 Concepts Reference

Concept / Term	Standard Definition PDF	HARM Framework Application PDF+ 1
Functional Safety	Safety depending on correct system responses to inputs.	Ensures motor commands strictly respect physical workspace boundaries.
Systematic Failures	Errors originating from software, logic, or specification flaws.	Treats VLM hallucinations and network latency stalls as systematic hazards.
Safety Integrity Level (SIL)	Target risk reduction measure (SIL 1 to SIL 4).	Defines the safety performance baseline for HARM's deterministic limit checks.
Fail-Safe Default	Automatic transition to a safe state upon fault detection.	Triggers zero-torque HOLD or velocity clamping when trajectory limits are breached.